LAPORAN PRAKTIKUM MOBILE PROGRAMMING

Laporan ini disusun untuk memenuhi tugas
Praktikum Mobile Programming Modul 13
Operasi CRUD di Flutter dengan REST API

Dosen Pengampu:

Khadijah Fahmi Hayati Holle, S.Kom., M.Kom

Disusun oleh:

Muhammad Rizqullah Saputra
240605110102

PRODI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI
MAULANA MALIK IBRAHIM MALANG

2026

TUJUAN

Praktikum ini menerapkan komunikasi client-server pada Flutter melalui operasi POST, PUT, dan DELETE ke JSONPlaceholder. Aplikasi menerima input Nama dan Pekerjaan, memetakan input ke title dan body, lalu menampilkan respons API dalam satu Card.

1. Memahami Create, Read, Update, Delete dan pemetaan metode HTTP dalam komunikasi REST API.

2. Mengirim request melalui http serta mengelola operasi Future menggunakan async dan await.

3. Mengubah JSON respons menjadi objek Post dan mengelola state createdPost.

4. Membuat AlertDialog untuk tambah, update, dan konfirmasi hapus serta menampilkan hasil melalui Card dan SnackBar.

5. Memahami peran setState() dalam pembaruan UI dan batas simulasi layanan JSONPlaceholder.

Lingkungan pengujian

Pengujian dilakukan pada Flutter 3.47.1, Dart 3.13.1, dan emulator Medium Phone Android API 37 dengan resolusi 1080 × 2400. Pengujian emulator dilakukan melalui otomasi pada aplikasi Android yang benar-benar dijalankan. Tanggal pengujian bukan klaim tanggal pertemuan praktikum di kelas.

JSONPlaceholder merupakan API pengujian. Respons create, update, dan delete bersifat simulasi; keberhasilan respons tidak berarti data tersimpan permanen pada server. Data Nama dan Pekerjaan yang digunakan merupakan data contoh.

LANGKAH KERJA SINGKAT

A Membuat proyek

Buat proyek flutter_crud_api; pada repository ini ditempatkan di langkah_kerja.

B Struktur

Kenali android/, lib/main.dart, dan pubspec.yaml.

C Dependency

Tambahkan http, intl, google_fonts, lalu jalankan flutter pub get.

D Import

Import Material, http dengan alias http, dart:convert, intl, dan Google Fonts.

E Aplikasi utama

MyApp mengatur MaterialApp, tema indigo, font Montserrat, halaman HomePage, dan debugShowCheckedModeBanner false.

F Model Post

Definisikan id, userId, title, body, createdAt dan factory Post.fromJson. userId default 1; createdAt dari DateTime.now().

G HomePage

Gunakan StatefulWidget dengan Post? createdPost, teks awal, Scaffold/AppBar, dan FloatingActionButton.

H createPost

Kirim POST /posts dengan header JSON dan payload title/body/userId 1; status 201 mengisi createdPost dari respons.

I Dialog tambah

Buat showAddPostDialog dengan TextField Nama dan Pekerjaan, controller, trim, validasi wajib isi, Batal dan Simpan.

J Hubungkan FAB

onPressed pada FAB memanggil showAddPostDialog.

K Tampilan Card

Jika createdPost null, tampilkan teks awal. Selain itu tampilkan Card berisi title, body, id, userID, dan tanggal.

L Coba tambah

Jalankan aplikasi, tekan +, isi Nama/Pekerjaan, Simpan, amati Card.

M updatePost

Sesuai modul, PUT diarahkan ke /posts/1 dan payload id 1. Status 200 memperbarui objek sambil mempertahankan createdAt.

N Dialog update

showUpdatePostDialog mengisi controller dengan data lama; validasi lalu jalankan updatePost.

O Menu pada Card

Judul dibungkus Expanded dalam Row bersama PopupMenuButton berisi Update dan Delete.

P Coba update

Tambahkan data, buka menu tiga titik, pilih Update, ubah isian, lalu amati Card dan notifikasi.

Q deletePost

Kirim DELETE /posts/$id; status 200 mengatur createdPost null dan menampilkan SnackBar hijau; gagal/error merah.

R Konfirmasi hapus

Buat showDeleteConfirmationDialog; Batal hanya menutup dialog, Hapus menutup dialog dan memanggil deletePost.

S Hubungkan menu Delete

Lengkapi onSelected agar Delete membuka konfirmasi untuk id post aktif.

T Coba alur lengkap

Jalankan create, tampilan/read, update, delete; ambil screenshot awal, dialog, Card, dan notifikasi sukses/gagal.

PEMBAHASAN KODE LANGKAH KERJA

Model JSON dan state createdPost

```
factory Post.fromJson(Map<String, dynamic> json) => Post(
  id: json['id'], userId: json['userId'] ?? 1,
  title: json['title'], body: json['body'],
  createdAt: DateTime.now(),
);
```

Post menyusun respons menjadi data Dart bertipe. createdPost bernilai null sebelum ada data dan setelah penghapusan; saat terisi, objek ditampilkan dalam Card. State ini menyimpan satu post aktif. createdAt adalah waktu lokal pembuatan model dan dipertahankan saat update. Literal WIB pada format tanggal tidak melakukan konversi zona waktu.

Create dan pembacaan respons

```
final response = await http.post(
  Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  headers: {'Content-type': 'application/json; charset=UTF-8'},
  body: jsonEncode({'title': title, 'body': body, 'userId': 1}),
);
```

createPost() memeriksa status 201, membaca jsonDecode(response.body), lalu mengisi createdPost menggunakan setState(). Read pada contoh modul adalah penampilan objek respons tersebut. Tidak terdapat request GET tersendiri pada langkah implementasinya. Dialog tambah memvalidasi dua isian setelah trim; data kosong tidak dikirim.

PEMBAHASAN KODE LANJUTAN

Update dan Delete

```
// Langkah m PDF menggunakan ID 1 untuk demo.
final url = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');
// Delete memakai ID post yang aktif.
final deleteUrl = Uri.parse(
  'https://jsonplaceholder.typicode.com/posts/$id',
);
```

updatePost() mengirim PUT dan menerima status 200 sebagai keberhasilan. Sesuai modul, URL dan id payload ditetapkan 1 sehingga id pada Card berubah dari respons create menjadi 1 setelah update. deletePost() menggunakan id aktif pada URL DELETE. Setelah respons 200, createdPost diatur null dan UI kembali ke awal. Dialog konfirmasi memungkinkan pengguna membatalkan penghapusan sebelum request dikirim.

Dialog dan notifikasi

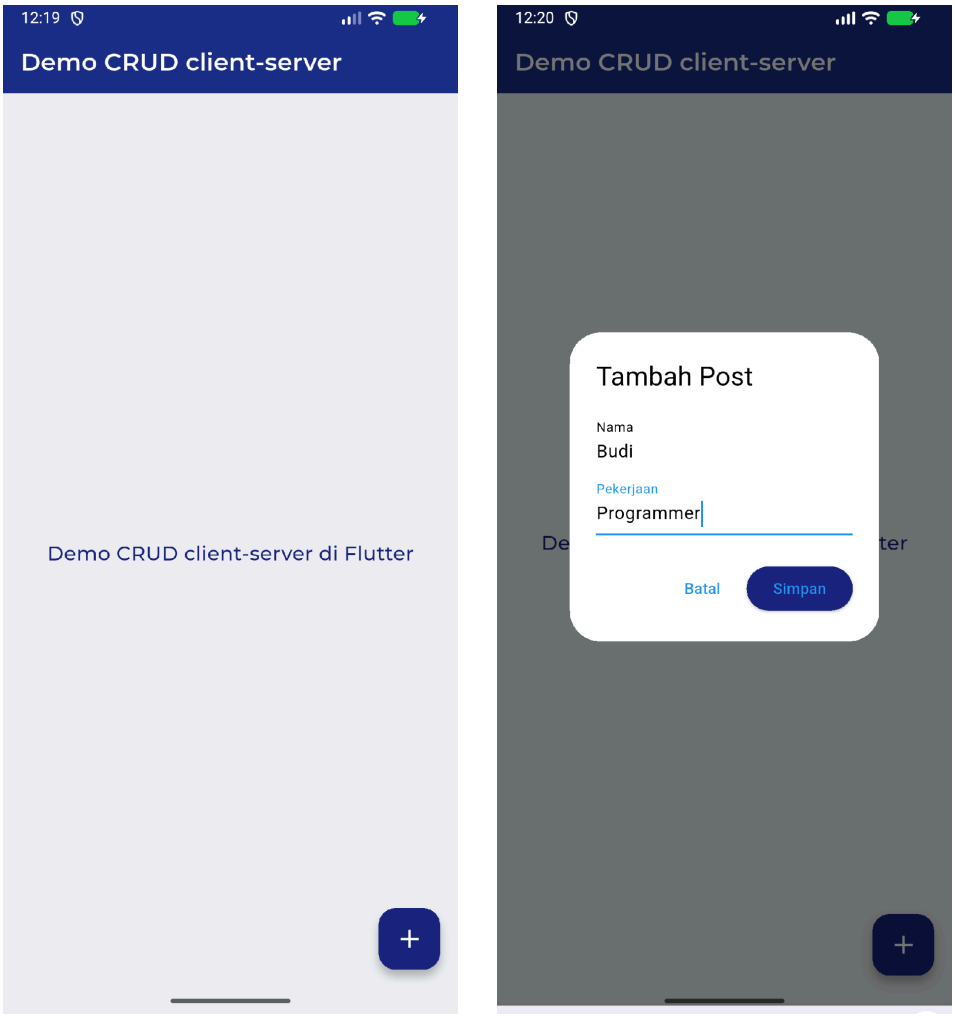
showAddPostDialog() menyediakan Nama/Pekerjaan; showUpdatePostDialog() mengisi form dengan data lama; showDeleteConfirmationDialog() menyediakan Batal/Hapus. PopupMenuButton menghubungkan opsi Update dan Delete. SnackBar sukses berwarna hijau, sedangkan error berwarna merah. try-catch menangani kegagalan request dan pemeriksaan mounted dilakukan sebelum menggunakan state atau context sesudah await.

Penyesuaian agar kompatibel

Dependency transitif path_provider_android 2.2.15 dan path_provider_foundation 2.4.1 dikunci melalui dependency_overrides untuk menghindari kegagalan hook native pada path Windows yang mengandung spasi. pubspec.lock menyimpan versi yang dipakai sehingga lingkungan dapat direproduksi.

Penomoran 12.2 sampai 12.5 pada PDF Modul XIII diperlakukan sebagai kesalahan penomoran, bukan perpindahan ke Modul XII. Warna error dibuat merah sesuai ketentuan screenshot. withValues(alpha: 0.3) menggantikan pemanggilan withOpacity(0.3) yang sudah deprecated dengan hasil warna yang sama.

SCREENSHOT



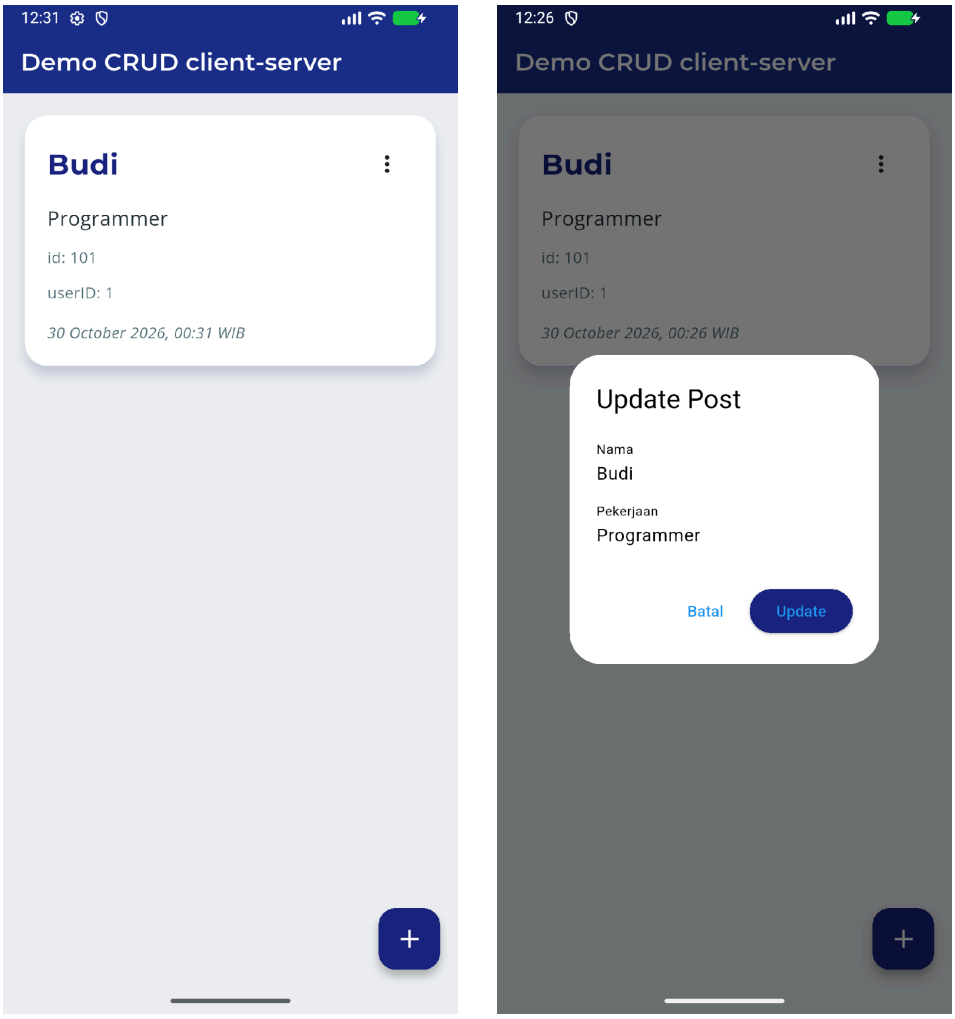
Gambar 1 Tampilan awal

createdPost masih null sehingga halaman menampilkan teks awal dan FAB tambah.

Gambar 2 Dialog tambah

Dialog menerima Nama dan Pekerjaan sebagai title dan body untuk request POST.

SCREENSHOT LANJUTAN



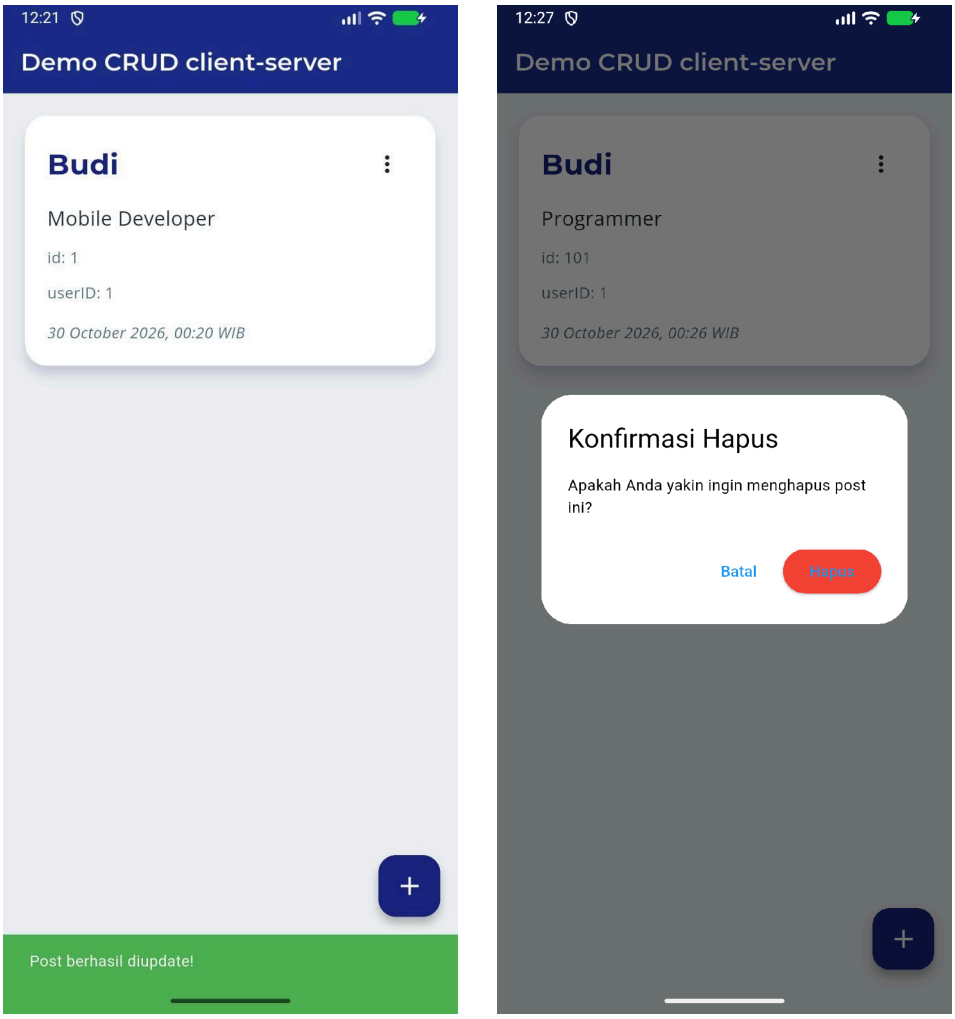
Gambar 3 Hasil create dalam Card

Respons POST ditampilkan sebagai satu Card dengan data contoh, id, userID, dan waktu lokal.

Gambar 4 Dialog update

Form update terisi data post yang sedang ditampilkan sehingga pengguna dapat memperbaruinya.

SCREENSHOT LANJUTAN



Gambar 5 Update berhasil

Data Card berubah setelah respons PUT berhasil. SnackBar hijau memberi umpan balik dan id mengikuti demo /posts/1.

Gambar 6 Konfirmasi penghapusan

Dialog meminta keputusan sebelum DELETE dikirim; Batal hanya menutup dialog.

SCREENSHOT LANJUTAN



Gambar 7 Delete berhasil

SnackBar hijau menandakan respons DELETE berhasil dan createdPost kembali null.

Gambar 8 Error jaringan

Koneksi jaringan emulator diputus sementara untuk memeriksa penanganan kegagalan request. SnackBar merah menampilkan pesan error aktual.

KESIMPULAN

1. Paket http menghubungkan aplikasi Flutter dengan JSONPlaceholder melalui POST, PUT, dan DELETE serta respons JSON.

2. Model Post memetakan data respons, sedangkan createdPost mengontrol apakah UI menampilkan teks awal atau Card.

3. AlertDialog menyediakan alur tambah, update, dan konfirmasi hapus. setState() memperbarui tampilan setelah respons operasi diterima.

4. Status respons dan kegagalan jaringan perlu ditangani. Screenshot memperlihatkan alur yang dijalankan serta notifikasi sukses dan error.

5. JSONPlaceholder hanya menyimulasikan perubahan server. Update ID 1 dan pembacaan respons pada state mengikuti ruang lingkup contoh modul.

Hasil pemeriksaan

Analyzer tidak menemukan masalah; dua tes parsing Post lulus. APK debug berhasil dibangun dan alur tambah, update, konfirmasi hapus, delete, serta error jaringan diperiksa pada emulator. Tidak ada pengujian database persisten atau perangkat iOS.

Sumber

Modul 13 praktikum yang diberikan, bagian Ringkasan, Langkah Kerja, Tugas, dan Format Laporan. Source implementasi: MODUL_13/langkah_kerja/lib/main.dart. Catatan pemeriksaan dan screenshot asli disertakan pada folder modul.

Panduan resmi JSONPlaceholder: https://jsonplaceholder.typicode.com/guide/ (diakses 30 Oktober 2026).